

LAMP Stack Deployment and SELinux Configuration

Server Engineering Labs

Author:

Laura Ortiz Moreno

Topics Covered:

- Apache HTTP Server
- MariaDB database server
- PHP integration
- Linux service management
- SELinux configuration
- Web application deployment

Environment:

Debian Linux Server
Rocky Linux Client VM
Virtualized lab environment

Contents

1	Introduction	2
2	Installing Apache Web Server	2
2.1	Verification	2
3	Installing MariaDB	2
3.1	Basic Database Security	3
4	Installing PHP	3
4.1	PHP Validation	3
5	Database Connectivity Testing	3
6	SELinux Configuration and Troubleshooting	4
6.1	SELinux Boolean Configuration	5
6.2	Firewall Verification	5
7	Service Validation	5
8	Challenges Encountered	6
9	Conclusion	6

1. Introduction

This report documents the deployment and validation of a LAMP environment in Linux.

The laboratory focused on:

- Installing Apache, MariaDB, and PHP
- Configuring system services
- Deploying a basic web application
- Verifying database connectivity
- Troubleshooting SELinux-related access restrictions

The setup was performed in a virtualized client-server environment using Linux virtual machines.

2. Installing Apache Web Server

The Apache HTTP server was installed using the Debian package manager.

Installation command:

```
sudo apt update  
sudo apt install apache2
```

The service status was verified:

```
sudo systemctl status apache2
```

Apache was configured to start automatically on boot:

```
sudo systemctl enable apache2
```

2.1. Verification

The server was tested from a client machine using a web browser.

Successful deployment was confirmed when the Apache default page became accessible.

3. Installing MariaDB

MariaDB was installed to provide relational database support.

Installation:

```
sudo apt install mariadb-server
```

The service was started and enabled:

```
sudo systemctl enable mariadb  
sudo systemctl start mariadb
```

Verification:

```
sudo systemctl status mariadb
```

3.1. Basic Database Security

The database server was secured using:

```
sudo mysql_secure_installation
```

Typical configuration tasks included:

- Removing anonymous users
- Disabling remote root login
- Removing test databases
- Enforcing secure credentials

4. Installing PHP

PHP support was installed to allow dynamic content generation.

Packages installed:

```
sudo apt install php libapache2-mod-php php-mysql
```

The Apache service was restarted:

```
sudo systemctl restart apache2
```

4.1. PHP Validation

A test file was created inside the Apache web root:

```
sudo nano /var/www/html/info.php
```

Content:

```
<?php
phpinfo();
?>
```

The page was then accessed from the browser to verify PHP integration.

5. Database Connectivity Testing

A simple database connection test was performed using PHP.

Example:

```
<?php
$conn = new mysqli("localhost", "user", "password", "database");

if ($conn->connect_error) {
    die("Connection failed");
}

echo "Database connection successful";
?>
```

The objective was to verify:

- PHP-MariaDB integration
- Database service accessibility

- Web application functionality

```
"index.html" 21L, 223B written
[www@localhost html]$ ls
index.html
[www@localhost html]$ curl http://11.111.110
<html>
<body>
Soy Rocky!!!

<?php

$link = mysqli_connect('127.0.0.1','root','_');

if(!$link){
    die ("No se puede conectar");
}

echo "Conectado a la base de datos";
mysqli_close($link);

?>

</body>
</html>
```

Figure 1: Validation of Apache, PHP, and MariaDB integration using a simple PHP database connection test.

6. SELinux Configuration and Troubleshooting

Part of the laboratory focused on understanding how SELinux affects services and filesystem access.

Useful commands included:

```
getenforce
sestatus
```

SELinux contexts were inspected using:

```
ls -Z
```

When required, contexts were restored using:

```
restorecon -Rv /var/www/html
```

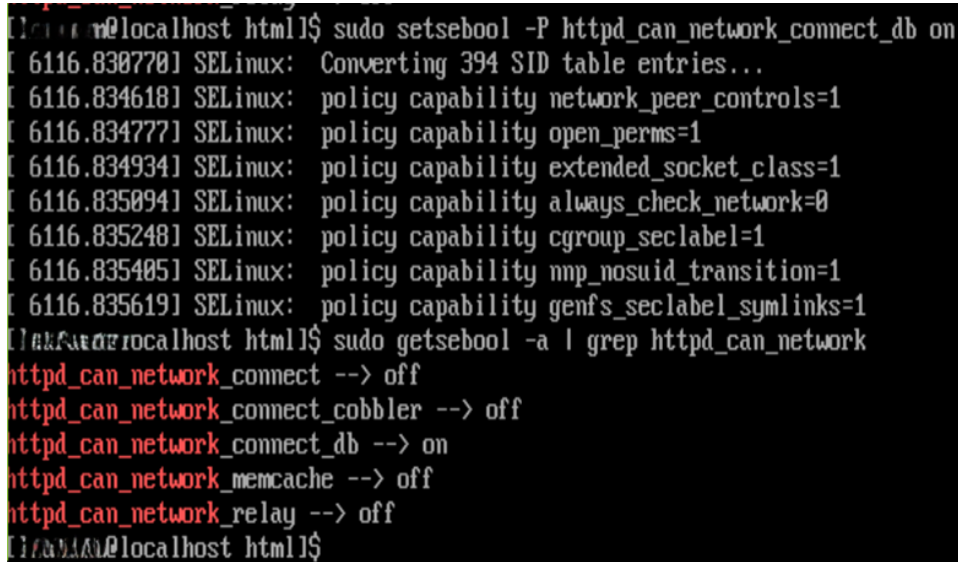
6.1. SELinux Boolean Configuration

SELinux booleans were modified to allow Apache to communicate with the database service.

```
sudo setsebool -P httpd_can_network_connect_db on
```

The configuration was validated using:

```
sudo getsebool -a | grep httpd_can_network
```



```
root@localhost html1$ sudo setsebool -P httpd_can_network_connect_db on
[ 6116.830770] SELinux: Converting 394 SID table entries...
[ 6116.834618] SELinux: policy capability network_peer_controls=1
[ 6116.834777] SELinux: policy capability open_perms=1
[ 6116.834934] SELinux: policy capability extended_socket_class=1
[ 6116.835094] SELinux: policy capability always_check_network=0
[ 6116.835248] SELinux: policy capability cgroup_seclabel=1
[ 6116.835405] SELinux: policy capability mmp_nosuid_transition=1
[ 6116.835619] SELinux: policy capability genfs_seclabel_symlinks=1
root@localhost html1$ sudo getsebool -a | grep httpd_can_network
httpd_can_network_connect --> off
httpd_can_network_connect_cobbler --> off
httpd_can_network_connect_db --> on
httpd_can_network_memcache --> off
httpd_can_network_relay --> off
root@localhost html1$
```

Figure 2: SELinux boolean configuration allowing Apache HTTP services to connect to the database backend.

6.2. Firewall Verification

Firewall rules were also validated to ensure HTTP traffic was allowed.

Example:

```
sudo ufw allow 80/tcp
sudo ufw reload
```

7. Service Validation

The environment was tested from external clients to validate:

- Apache availability
- PHP execution
- Database connectivity
- Firewall accessibility

The complete LAMP stack operated correctly after service configuration and troubleshooting.

8. Challenges Encountered

Several issues appeared during deployment:

- Apache service conflicts
- PHP module configuration errors
- Database permission problems
- SELinux access denials
- Firewall restrictions

The exercises provided practical troubleshooting experience in Linux server administration.

9. Conclusion

This laboratory introduced the deployment and administration of a complete LAMP environment.

The exercises demonstrated:

- Linux web server deployment
- Database administration basics
- PHP integration
- SELinux troubleshooting
- Service validation and monitoring

These concepts are essential in backend infrastructure, Linux administration, and web server management.